Kasi Yedumati · Full Stack Training

Lesson 5 — Database Connectivity

Course: FastAPI for Full Stack · **Session 5 of 10** · Learning path · **Previous:** Lesson 4 · **Next:** Lesson 6

> **Module Outcome:** You can create, read, update, and delete the AcmeHR employees in `training_db` through a route, a service, and a repository: first with SQL you write through `mysql-connector-python`, then with SQLAlchemy. You can show that only the repository changed.

You will build	`GET`, `POST`, `PATCH`, and `DELETE` on `/employees` against MySQL instead of a Python list, first with a cursor, then with a session
New words	Connection, cursor, commit, repository, service, engine, session, table model
You need	MySQL running, database `training_db`, and `EmployeeCreate`, `EmployeeUpdate`, and `EmployeeOut` from Lessons 3 and 4

From the requirements. Employee records are the source of truth for the directory and the profile. A list inside `main.py` disappears when `fastapi dev` stops. The rows belong in MySQL.

The practice login is the one from the MySQL setup lesson: user `trainee`, password `password123`, database `training_db`. If your password differs, set it in the terminal before `fastapi dev`: `DB_PASSWORD` for step 1, `DATABASE_URL` for step 2.

1. The table and the six rows

Run this once in MySQL.

```
USE training_db;

CREATE TABLE IF NOT EXISTS employee (
  id INT PRIMARY KEY AUTO_INCREMENT,
  employee_id VARCHAR(10) NOT NULL UNIQUE,
  name VARCHAR(80) NOT NULL,
  designation VARCHAR(80) NOT NULL,
  department VARCHAR(80) NOT NULL,
  email VARCHAR(120) NOT NULL,
  phone VARCHAR(20) NULL
);

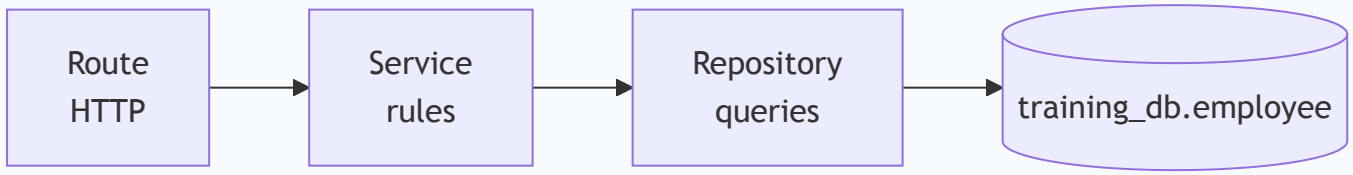
INSERT IGNORE INTO employee (employee_id, name, designation, department, email, phone) VALUES
  ('E-1042', 'Ananya Rao', 'Senior Software Engineer', 'Engineering', 'ananya.rao@acme.example', '+91 9(
  ('E-1043', 'Karthik Menon', 'Engineering Manager', 'Engineering', 'karthik.menon@acme.example', NULL),
  ('E-1044', 'Meera Iyer', 'Accountant', 'Finance', 'meera.iyer@acme.example', NULL),
  ('E-1045', 'Rahul Sharma', 'HR Executive', 'Human Resources', 'rahul.sharma@acme.example', NULL),
  ('E-1046', 'Fatima Khan', 'Product Designer', 'Product', 'fatima.khan@acme.example', NULL),
  ('E-1047', 'Arjun Reddy', 'QA Engineer', 'Engineering', 'arjun.reddy@acme.example', NULL);
```

`id` is MySQL's number. `employee_id` is the HR id, `E-1042`. The URL keeps using `E-1042`. `INSERT IGNORE` skips a row whose `employee_id` is already in the table, so running the block twice does not add a second Ananya.

---

2. Three layers

A route could open MySQL, run the SQL, and decide on 404 all by itself. From this lesson on, that work is split three ways.



Layer	Its job	It never
Route	Reads the URL, opens and closes the database, picks `response_model`	Runs SQL
Service	The rules. "There is no such employee" is a rule. It commits a change when the work is done	Runs SQL
Repository	The queries. Returns rows, or `None`	Decides what a missing row means

Each layer calls only the one below it. Step 1 writes the repository with a cursor. Step 2 rewrites the repository with SQLAlchemy, and the service does not change.

---

3. Step 1 — the repository runs SQL through a cursor

This is the driver from Database Programming: connect, open a cursor, `execute` with `%s`, fetch, close.

```
pip install mysql-connector-python
```

The whole file is `examples/lesson05/01_connector.py`. `EmployeeCreate`, `EmployeeUpdate`, `EmployeeOut`, `/`, and `/health` are the same as in Lesson 4. Reads come first, then create, update, and delete.

The repository holds every SQL string.

```
import mysql.connector

class EmployeeRepository:
    def __init__(self, db):
        self.db = db

    def get_all(self, department=None):
        cursor = self.db.cursor(dictionary=True)
        try:
            if department is None:
                cursor.execute(
                    "SELECT employee_id, name, designation, department, email, phone "
                    "FROM employee"
                )
            else:
```

```
                cursor.execute(
                    "SELECT employee_id, name, designation, department, email, phone "
                    "FROM employee WHERE department = %s",
                    (department,),
                )
            return cursor.fetchall()
        finally:
            cursor.close()

    def get(self, employee_id):
        cursor = self.db.cursor(dictionary=True)
        try:
            cursor.execute(
                "SELECT employee_id, name, designation, department, email, phone "
                "FROM employee WHERE employee_id = %s",
                (employee_id,),
            )
            return cursor.fetchone()
        finally:
            cursor.close()
```

Line	Job
`self.db`	The open MySQL connection. The route hands it in
`cursor(dictionary=True)`	Each row comes back as a dict, `{"employee_id": "E-1042", ...}`, which `EmployeeOut` can read
`%s` and `(department,)`	The value travels separately from the SQL. Never build this SQL with an f-string
`fetchall()`	A list of rows. An unknown department gives `[]`
`fetchone()`	One row, or `None`. The repository does not decide what `None` means
`finally`	Closes the cursor, even when the query fails

The service turns "no row" into **404**, with the `HTTPException` from Lesson 4.

```
from fastapi import HTTPException

class EmployeeService:
    def __init__(self, db):
        self.db = db
        self.repo = EmployeeRepository(db)

    def get_all(self, department=None):
        return self.repo.get_all(department)

    def get(self, employee_id):
        row = self.repo.get(employee_id)
        if row is None:
            raise HTTPException(status_code=404, detail="Employee not found")
        return row
```

`get_all` only passes the call on. The service keeps `self.db` for `commit`, which create, update, and delete below need.

`HTTPException` does not need to be raised in the route. FastAPI catches it wherever it was raised, here inside the service, and sends `{"detail": "Employee not found"}` with **404**. Lesson 6 replaces it with an error the service names itself, and shows why.

The routes are the HTTP layer.

```
@app.get("/employees", response_model=list[EmployeeOut])
def list_employees(department: str | None = None):
    db = mysql.connector.connect(**DB_CONFIG)
    try:
        return EmployeeService(db).get_all(department)
    finally:
        db.close()

@app.get("/employees/{employee_id}", response_model=EmployeeOut)
def get_employee(employee_id: str):
    db = mysql.connector.connect(**DB_CONFIG)
    try:
        return EmployeeService(db).get(employee_id)
    finally:
        db.close()
```

Piece	Job
`DB_CONFIG`	Host, port, user, password, and database, at the top of the file. `**DB_CONFIG` passes them to `connect`
`EmployeeService(db).get(...)`	The route hands the connection to the service and returns what it gets back
`finally: db.close()`	Closes the connection, even when the service raises 404

Start the app and open `GET /employees/E-1042`.

Output:

```
{
  "employee_id": "E-1042",
  "name": "Ananya Rao",
  "designation": "Senior Software Engineer",
  "department": "Engineering",
  "email": "ananya.rao@acme.example",
  "phone": "+91 90000 01042"
}
```

`GET /employees?department=Engineering` returns E-1042, E-1043, and E-1047. `GET /employees/E-9999` is **404** with `{"detail":"Employee not found"}`.

Common error: `self.db.cursor()` without `dictionary=True`. Each row is a tuple, `('E-1042', 'Ananya Rao', ...)`. The browser shows **500** `Internal Server Error`, and the terminal shows `ResponseValidationError` with `Input should be a valid dictionary or object to extract fields from`.

Common error: `(department)` without the comma. That is a string, not a tuple. The terminal shows `ProgrammingError: Could not process parameters: str(Engineering), it must be of type list, tuple or dict`.

Common error: GET `E-9999` is **500**, and the terminal shows `ResponseValidationError` with `Input should be a valid dictionary or object to extract fields from` and `input: None`. The service returned `row` without checking for `None`. Raise the 404 before `return row`.

Create, update, and delete

POST registers a new employee. PATCH changes only the fields the caller sends. DELETE removes the row. Each layer gains three methods.

The repository writes the SQL. `add` takes a dict of columns. `update` merges the changes into the row it was given, then writes every column back:

```
def add(self, values):
        cursor = self.db.cursor()
        try:
            cursor.execute(
                "INSERT INTO employee (employee_id, name, designation, department, email, phone) "
                "VALUES (%s, %s, %s, %s, %s, %s)",
                (values["employee_id"], values["name"], values["designation"], values["department"], va
            )
        finally:
            cursor.close()

    def update(self, row, values):
        row.update(values)
        cursor = self.db.cursor()
        try:
            cursor.execute(
                "UPDATE employee SET name = %s, designation = %s, department = %s, email = %s, phone = 
                "WHERE employee_id = %s",
                (row["name"], row["designation"], row["department"], row["email"], row["phone"], row["er
            )
        finally:
            cursor.close()
        return row

    def delete(self, row):
        cursor = self.db.cursor()
        try:
            cursor.execute("DELETE FROM employee WHERE employee_id = %s", (row["employee_id"],))
        finally:
            cursor.close()
```

Every value still travels as `%s`. A column that was not sent keeps the value it already had in `row`.

The service checks before it writes. `register` refuses an id that is already taken. `update` and `delete` find the employee first, so an unknown id is **404** before any SQL runs. Then each one commits:

```
def register(self, data):
        if self.repo.get(data.employee_id) is not None:
            raise HTTPException(status_code=400, detail="Employee id already exists")
        self.repo.add(data.model_dump())
        self.db.commit()
        return self.get(data.employee_id)
```

```
    def update(self, employee_id, changes):
        row = self.get(employee_id)
        self.repo.update(row, changes.model_dump(exclude_unset=True))
        self.db.commit()
        return self.get(employee_id)

    def delete(self, employee_id):
        row = self.get(employee_id)
        self.repo.delete(row)
        self.db.commit()
```

Line	Job
`self.repo.get(data.employee_id) is not None`	The id is taken. **400**, the same rule Lesson 3 wrote against the Python list
`data.model_dump()`	The checked `EmployeeCreate` as a dict. The repository writes those six columns
`self.get(employee_id)`	The same method GET uses. A missing employee is 404 for PATCH and DELETE too
`model_dump(exclude_unset=True)`	Only the keys the caller sent. A PATCH of the designation does not touch the email
`self.db.commit()`	Makes the change permanent. Without it, MySQL throws the change away when the connection closes
`return self.get(...)`	Reads the row again, so the response shows what MySQL now holds

The repository changes rows. The service decides when the work is finished and commits.

The routes have the same shape as the GET routes:

```
from fastapi import Response

@app.post("/employees", status_code=201, response_model=EmployeeOut)
def create_employee(employee: EmployeeCreate):
    db = mysql.connector.connect(**DB_CONFIG)
    try:
        return EmployeeService(db).register(employee)
    finally:
        db.close()

@app.patch("/employees/{employee_id}", response_model=EmployeeOut)
def update_employee(employee_id: str, changes: EmployeeUpdate):
    db = mysql.connector.connect(**DB_CONFIG)
    try:
        return EmployeeService(db).update(employee_id, changes)
    finally:
        db.close()

@app.delete("/employees/{employee_id}", status_code=204)
def delete_employee(employee_id: str):
    db = mysql.connector.connect(**DB_CONFIG)
    try:
        EmployeeService(db).delete(employee_id)
        return Response(status_code=204)
```

```
    finally:
        db.close()
```

POST Sana:

```
{
  "employee_id": "E-1048",
  "name": "Sana Qureshi",
  "designation": "HR Executive",
  "department": "Human Resources",
  "email": "sana.qureshi@acme.example"
}
```

201, and the body comes back with `"phone": null`. `SELECT * FROM employee WHERE employee_id = 'E-1048';` shows her row. Send the same body again: **400** and `Employee id already exists`. A bad email is still **422** from `EmployeeCreate`, and the service never runs.

PATCH `E-1046` with `{"designation": "Senior Product Designer"}`.

Output:

```
{
  "employee_id": "E-1046",
  "name": "Fatima Khan",
  "designation": "Senior Product Designer",
  "department": "Product",
  "email": "fatima.khan@acme.example",
  "phone": null
}
```

DELETE `E-1047` is **204** with an empty body. GET `E-1047` is now **404**. Run the `INSERT IGNORE` from section 1 again to bring Arjun back.

Common error: POST or PATCH answers with the new data, but a `SELECT` in MySQL does not show it. The service does not call `self.db.commit()`.

Common error: PATCH with `{"name": null}` is **422**. That is `not_null` in `EmployeeUpdate` from Lesson 4, working as intended. `phone` may be `null`, which clears it.

---

4. Step 2 — SQLAlchemy replaces the repository

Step 1 works. Look at what the repository typed by hand.

In step 1 you	In step 2 SQLAlchemy
Typed the column list in every `SELECT`	Reads the columns from one `Employee` class
Wrote two SQL strings, with and without `WHERE`	Adds `.filter(...)` only when a department is given
Ran `CREATE TABLE` and the `INSERT` in MySQL	Creates the table and inserts the six people when the server starts
Got dicts back	Gets `Employee` objects back

The driver stays. SQLAlchemy writes the SQL and hands it to `mysql.connector`.

```
pip install sqlalchemy
```

The whole file is `examples/lesson05/02_sqlalchemy.py`. Replace the step 1 code in `main.py` with it.

```
import os
from sqlalchemy import Column, Integer, String, create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "mysql+mysqlconnector://trainee:password123@localhost:3306/training_db",
)
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
Base = declarative_base()

class Employee(Base):
    __tablename__ = "employee"

    id = Column(Integer, primary_key=True)
    employee_id = Column(String(10), unique=True, nullable=False)
    name = Column(String(80), nullable=False)
    designation = Column(String(80), nullable=False)
    department = Column(String(80), nullable=False)
    email = Column(String(120), nullable=False)
    phone = Column(String(20), nullable=True)
```

Name	What it is
`DATABASE_URL`	Where MySQL is. `mysql+mysqlconnector` tells SQLAlchemy to use the step 1 driver
`engine`	The connection machinery
`SessionLocal`	A factory. Call it to get one session for one request
`Employee`	The table, written as a Python class. This is not `Employee Out`

The repository keeps its method names. Only the bodies change.

```
class EmployeeRepository:
    def __init__(self, db):
        self.db = db

    def get_all(self, department=None):
        query = self.db.query(Employee)
        if department is not None:
            query = query.filter(Employee.department == department)
        return query.all()

    def get(self, employee_id):
        return self.db.query(Employee).filter(Employee.employee_id == employee_id).one_or_none()

    def add(self, values):
        self.db.add(Employee(**values))
```

```
    def update(self, row, values):
        for key, value in values.items():
            setattr(row, key, value)
        return row

    def delete(self, row):
        self.db.delete(row)
```

`self.db` is now a session, not a connection. `one_or_none()` is the session's `fetchone()`: one row, or `None`. `add` turns the dict into an `Employee` and stages it. `update` sets attributes on the `Employee` object. `delete` marks the row. SQLAlchemy writes the `INSERT`, `UPDATE`, or `DELETE` when the service commits.

`add` still takes a dict, as in step 1. The service passes `data.model_dump()` and never sees the `Employee` class, so it does not change.

In each route, one line changes:

```
db = SessionLocal()                        # step 1: mysql.connector.connect(**DB_CONFIG)
```

All five `EmployeeService` methods, `get_all`, `get`, `register`, `update`, and `delete`, and the rest of each route are the step 1 code, unchanged. `self.db.commit()` works on a connection and on a session alike. That is what the layers buy you: changing how the data is fetched touched one class.

`EmployeeOut` is still the response model, with `from_attributes=True`, so an `Employee` object can be turned into JSON.

---

5. Insert on startup

When `fastapi dev` starts, create the table if it is missing and insert the six people if the table is empty. The repository gains `count`. The service gains `seed`, because "insert only into an empty table" is a rule. `seed` reuses `repo.add` from POST.

```
class EmployeeRepository:
    ...
    def count(self):
        return self.db.query(Employee).count()

class EmployeeService:
    ...
    def seed(self, people):
        if self.repo.count() > 0:
            return
        for employee_id, name, designation, department, email, phone in people:
            self.repo.add({
                "employee_id": employee_id,
                "name": name,
                "designation": designation,
                "department": department,
                "email": email,
                "phone": phone,
            })
        self.db.commit()
```

`SEED` in the example file is the list of six people. Ananya is the first tuple. `repo.add` stages one row per person. `self.db.commit()` writes all six in one go, the same commit that PATCH and DELETE use.

You already ran the `INSERT` in section 1, so on your machine `seed` finds six rows and adds nothing. A new laptop with an empty `training_db` gets the six people without any SQL typed by hand.

FastAPI calls `seed` from a `lifespan` function, which runs once at startup:

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(engine)
    db = SessionLocal()
    try:
        EmployeeService(db).seed(SEED)
    finally:
        db.close()
    yield

app = FastAPI(title="AcmeHR", lifespan=lifespan)
```

Line	Job
`Base.metadata.create_all(engine)`	Creates every table class that is missing. An existing table is left alone
Code before `yield`	Runs once, when the server starts
Code after `yield`	Runs once, when the server stops. Nothing is needed here
`lifespan=lifespan`	Tells FastAPI to use it

Copy this block as it is. You do not need to change `async` or `yield`. Today the job is: create the table, then seed.

Start the app and open `GET /employees/E-1042`. The JSON is the same as in step 1. Then in MySQL:

```
SELECT employee_id, name, department FROM employee;
```

You should see six rows. Restart `fastapi dev`. The six rows are still there. `seed` sees a count above zero and does not insert them again.

POST Sana, PATCH `E-1046`, and DELETE `E-1047` again, now through SQLAlchemy. The status codes and the JSON are the same as in step 1. If Sana is already in the table from step 1, the POST is **400**: the rule still works.

Common error: `Access denied for user 'trainee'`. The user or password does not match your MySQL. Set `DB_PASSWORD` (step 1) or `DATABASE_URL` (step 2) in the terminal before `fastapi dev`.

Common error: `Unknown database 'training_db'`. Create it: `CREATE DATABASE IF NOT EXISTS training_db;`

Common error: `Can't connect to MySQL server`. The MySQL service is not running.

Try these

1. **(Easy)** With step 1, GET `/employees?department=Finance`. You get Meera only.

2. **(Medium)** In step 1, change `self.db.cursor(dictionary=True)` to `self.db.cursor()` in `get`. GET `/employees/E-1042`. Read the terminal, then put it back.
3. **(Easy)** With step 1, POST a new employee `E-1050` without a phone. GET that id. `phone` is `null`. POST the same body again: **400**.
4. **(Easy)** With step 1, PATCH `E-1043` with `{"phone": "+91 90000 01043"}`. Run a `SELECT` in MySQL. Only `phone` changed.
5. **(Medium)** In step 1, comment out `self.db.commit()` in `EmployeeService.delete`. DELETE `E-1047` is **204**, but GET `E-1047` is still **200**. Put the commit back.
6. **(Easy)** With step 2, GET `/employees?department=Engineering`. You get Ananya, Karthik, and Arjun, the same as step 1.
7. **(Medium)** Put step 1 and step 2 side by side. List every line that differs in `EmployeeService` (except `seed`) and in the five routes. The service has none. Each route has one.
8. **(Medium)** `DELETE FROM employee;` in MySQL, restart `fastapi dev` with step 2, and GET the list. The six people are back. `seed` ran because the table was empty. Step 1 would return `[]`.
9. **(Stretch)** Add a seventh person with `INSERT INTO employee ...` and GET that id without changing Python. It works in both steps.

Interview: *Why split a route, a service, and a repository?* — Each has one reason to change. Moving from raw SQL to SQLAlchemy rewrote the repository and left the service and its rules alone. The route only speaks HTTP.

Interview: *Why use SQLAlchemy when `mysql-connector-python` already works?* — The driver runs the SQL you write. SQLAlchemy writes that SQL from a class, creates the table, and returns objects. It still uses the driver underneath.

Interview: *What is a session?* — One conversation with MySQL for one piece of work. Open it, run the query, commit if you changed rows, close it.

Key takeaways

- The route speaks HTTP. The service holds the rules. The repository holds the queries.
- The service raises **404** for a missing employee. FastAPI turns an `HTTPException` into JSON wherever it is raised.
- A cursor runs the SQL you write. `%s` with a tuple keeps the value out of the SQL string.
- SQLAlchemy uses the same driver. Switching to it rewrote only the repository.
- `Employee` is the table. `EmployeeOut` is the JSON.
- The repository changes rows. The service commits. Without `commit`, an insert, an update, or a delete is lost when the connection closes.
- `repo.add` takes a dict, so `register` is the same in both steps. A taken id is **400**.
- PATCH sends only the changed keys: `exclude_unset=True`. An unknown id is **404** for PATCH and DELETE, through the same `self.get`.
- The URL still uses `E-1042`, not the numeric `id`.

Lab — Ticket ACME-205

HR wants the six employees stored in `training_db`. The directory must read that table, and HR must be able to register, correct, or remove a record.

```
PYTHON/17_FastAPI/labs/L05_mysql/
├── broken/main.py      ← Part A. Fix this file.
├── starter/main.py     ← Part B. Complete the TODOs.
└── solution/main.py    ← Open only after the checklist passes.
```

MySQL must be running. Set `DATABASE_URL` in the terminal if your login is not `trainee` / `password123`. Section 1 already put six rows in the table. Before Part A and again before Part B, run `DROP TABLE IF EXISTS employee;` in MySQL, so `seed` starts from an empty table.

Part A — Fix the broken API (10 min)

`broken/main.py` starts, and it has **4 bugs**. Fix them in order. The first one hides the second. For each bug, decide first which layer owns it: the route, the service, or the repository.

1. After startup, `GET /employees` is `[]`, and `SELECT * FROM employee;` shows no rows.
2. Once rows are saved, restart `fastapi dev` again. The start crashes with `IntegrityError` and `Duplicate entry 'E-1042'`.
3. `GET /employees/E-1042` is **404**, although Ananya is in the table.
4. `GET /employees?department=Engineering` returns all six people.

Part B — Build it yourself (15 min)

`starter/main.py` has the connection, `SEED`, the three Pydantic models, the repository's `update` and `delete`, every service method except `seed`, and the `lifespan` function from section 5. Write the `Employee` class, the repository's `get_all`, `get`, `count`, and `add`, the body of `seed`, and the GET, POST, PATCH, and DELETE routes.

Part C — Prove it works (5 min)

- [ ] After `fastapi dev main.py`, MySQL shows six rows in `employee`.
- [ ] `GET /employees/E-1042` is Ananya and **200**.
- [ ] `GET /employees/E-9999` is **404**.
- [ ] `GET /employees?department=Engineering` is only E-1042, E-1043, and E-1047.
- [ ] Restart the server twice. The six rows are still there, and there are no duplicate Ananya rows.
- [ ] POST Sana as `E-1048` is **201**, and MySQL shows her row. The same POST again is **400**.
- [ ] PATCH `E-1046` with only a new designation is **200**. In MySQL her email is unchanged.
- [ ] DELETE `E-1047` is **204**. GET `E-1047` is then **404**, and MySQL has no row for him.
- [ ] PATCH and DELETE on `E-9999` are **404**.
- [ ] No route contains `query`, `filter`, or SQL. Only the repository does.

Run `DROP TABLE IF EXISTS employee;` and restart when you are done, so the table holds exactly the six people again for Lesson 6.

Open `solution/main.py` only after the checklist passes.

Homework

1. Finish the lab in `acmehr-api`. A full replace with PUT comes next, on this same table.
2. **Transfer drill (15 minutes).** In MySQL, `INSERT` one more employee and read that person with GET. Delete that row when you are done so the class seed stays the six people.

Next: CRUD. PUT completes the API, a duplicate id gets a rollback, and the service names its own errors.